

trainYOLOv4ObjectDetector

Train YOLO v4 object detector
Since R2022a

Syntax

```
detector = trainYOLOv4ObjectDetector(trainingData,detector,options)
detector = trainYOLOv4ObjectDetector(trainingData,checkpoint,options)
[detector,info] = trainYOLOv4ObjectDetector( __ )
__ = trainYOLOv4ObjectDetector( __ ,Name=Value)
```

Description

detector = trainYOLOv4ObjectDetector(trainingData,detector,options) returns an object detector trained using you only look once version 4 (YOLO v4) network specified by the input detector. The input detector can be an untrained or pretrained YOLO v4 object detector. example
The options input specifies training parameters for the detection network.

You can also use this syntax for fine-tuning a pretrained YOLO v4 object detector.

detector = trainYOLOv4ObjectDetector(trainingData,checkpoint,options) resumes training from the saved detector checkpoint.

You can use this syntax to:

- Add more training data and continue the training.
- Improve training accuracy by increasing the maximum number of iterations.

[detector,info] = trainYOLOv4ObjectDetector(__) also returns information on the training progress, such as the training accuracy and learning rate for each iteration.

__ = trainYOLOv4ObjectDetector(__ ,Name=Value) specifies options using one or more name-value arguments in addition to any combination of arguments from previous syntaxes. For example, trainYOLOv4ObjectDetector(trainingData,ExperimentManager="none") sets the metrics to track with Experiment Manger to "none".



Note

To run this function, you will require the Deep Learning Toolbox™.

Examples

collapse all

Train YOLO v4 Network for Vehicle Detection

This example shows how to fine-tune a pretrained YOLO v4 object detector for detecting vehicles in an image. This example uses a tiny YOLO v4 network trained on COCO dataset.

Load a pretrained YOLO v4 object detector and inspect its properties.

Try This Example

This example uses:
Computer Vision Toolbox
Deep Learning Toolbox

Copy Command

detector = yolov4ObjectDetector("tiny-yolov4-coco")

Get

detector =
yolov4ObjectDetector with properties:

Network: [1x1 dlnetwork]
AnchorBoxes: {2x1 cell}
ClassNames: {80x1 cell}
InputSize: [416 416 3]
ModelName: 'tiny-yolov4-coco'

The number of anchor boxes must be same as that of the number of output layers in the YOLO v4 network. The tiny YOLO v4 network contains two output layers.

detector.Network

Get

ans =
dlnetwork with properties:

Layers: [74x1 nnet.cnn.layer.Layer]
Connections: [80x2 table]
Learnables: [80x3 table]
State: [38x3 table]
InputNames: {'input_1'}
OutputNames: {'conv_31' 'conv_38'}
Initialized: 1

Prepare Training Data

Load a .mat file containing information about the vehicle dataset to use for training. The information stored in the .mat file is a table. The first column contains the training images and the remaining columns contain the labeled bounding boxes.

data = load("vehicleTrainingData.mat");
trainingData = data.vehicleTrainingData;

Get

Specify the directory in which training samples are stored. Add full path to the file names in training data.

dataDir = fullfile(toolboxdir('vision'),'visiondata');
trainingData.imageFilename = fullfile(dataDir,trainingData.imageFilename);

Get

Create an imageDatastore using the files from the table.

imds = imageDatastore(trainingData.imageFilename);

Get

Create a boxLabelDatastore using the label columns from the table.

blds = boxLabelDatastore(trainingData(:,2:end));

Get

Combine the datastores.

ds = combine(imds,blds);

Get

Specify the input size to use for resizing the training images. The size of the training images must be a multiple of 32 for when you use the tiny-yolov4-coco and csp-darknet53-coco pretrained YOLO v4 deep learning networks. You must also resize the bounding boxes based on the specified input size.

inputSize = [224 224 3];

Get

Resize and rescale the training images and the bounding boxes by using the preprocessData helper function. Also, convert the preprocessed data to a datastore object by using the transform function.

trainingDataForEstimation = transform(ds,@(data)preprocessData(data,inputSize));

Get

Estimate Anchor Boxes

Estimate the anchor boxes from the training data. You must assign the same number of anchor boxes to each output layer in the YOLO v4 network.

numAnchors = 6;
[anchors, meanIoU] = estimateAnchorBoxes(trainingDataForEstimation,numAnchors);
area = anchors(:,1).*anchors(:,2);
[~,idx] = sort(area,"descend");
anchors = anchors(idx,:);
anchorBoxes = {anchors(1:3,:);anchors(4:6,:)};

Get

Configure and Train YOLO v4 Network

Specify the class names and configure the pretrained YOLOv4 deep learning network to retrain for the new dataset by using yolov4ObjectDetector function.

classes = {'vehicle'};
detector = yolov4ObjectDetector("tiny-yolov4-coco",classes,anchorBoxes,InputSize=inputSize);

Get

Specify the training options and retrain the pretrained YOLO v4 network on the new dataset by using the trainYOLOv4ObjectDetector function.

options = trainingOptions("sgdm", ...
InitialLearnRate=0.001, ...
MiniBatchSize=16,...
MaxEpochs=40, ...
BatchNormalizationStatistics="moving",...
ResetInputNormalization=false,...
VerboseFrequency=30);
trainedDetector = trainYOLOv4ObjectDetector(ds,detector,options);

Training a YOLO v4 Object Detector for the following object classes:

* vehicle

Epoch	Iteration	TimeElapsed	LearnRate	TrainingLoss
2	30	00:01:07	0.001	7.215
4	60	00:01:44	0.001	1.7371
5	90	00:02:21	0.001	0.97954
7	120	00:02:57	0.001	0.59412
8	150	00:03:34	0.001	0.65631
10	180	00:04:10	0.001	1.0774
12	210	00:04:46	0.001	0.4807
13	240	00:05:22	0.001	0.40389
15	270	00:05:59	0.001	0.57931
16	300	00:06:35	0.001	0.90734
18	330	00:07:11	0.001	0.24902
19	360	00:07:48	0.001	0.32441
21	390	00:08:24	0.001	0.23054
23	420	00:09:00	0.001	0.70897
24	450	00:09:36	0.001	0.31744
26	480	00:10:12	0.001	0.36323
27	510	00:10:49	0.001	0.13696
29	540	00:11:25	0.001	0.14913
30	570	00:12:01	0.001	0.37757
32	600	00:12:37	0.001	0.36985
34	630	00:13:14	0.001	0.14034
35	660	00:13:50	0.001	0.14731
37	690	00:14:26	0.001	0.15907
38	720	00:15:03	0.001	0.11737
40	750	00:15:40	0.001	0.1855

Detector training complete.

Detect Vehicles in Test Image

Read a test image.

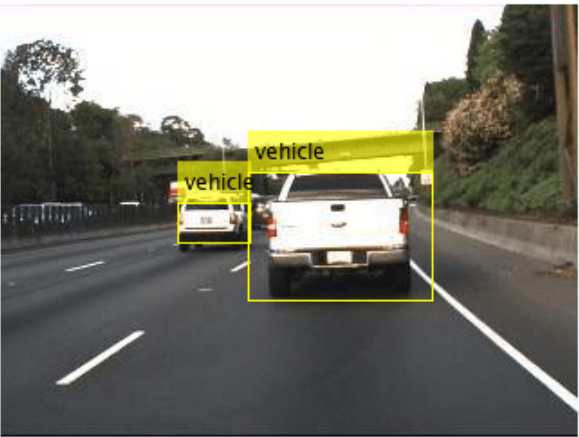
I = imread('highway.png');

Get

Use the fine-tuned YOLO v4 object detector to detect vehicles in a test image and display the detection results.

[bboxes, scores, labels] = detect(trainedDetector,I,Threshold=0.05);
detectedImg = insertObjectAnnotation(I,"Rectangle",bboxes,labels);
figure
imshow(detectedImg)

Get



function data = preprocessData(data,targetSize)
for num = 1:size(data,1)
I = data{num,1};
imgSize = size(I);
bboxes = data{num,2};
I = im2single(imresize(I,targetSize(1:2)));
scale = targetSize(1:2)./imgSize(1:2);
bboxes = bboxresize(bboxes,scale);
data(num,1:2) = {I,bboxes};
end
end

Get

Input Arguments

collapse all

trainingData — Labeled ground truth images
datastore

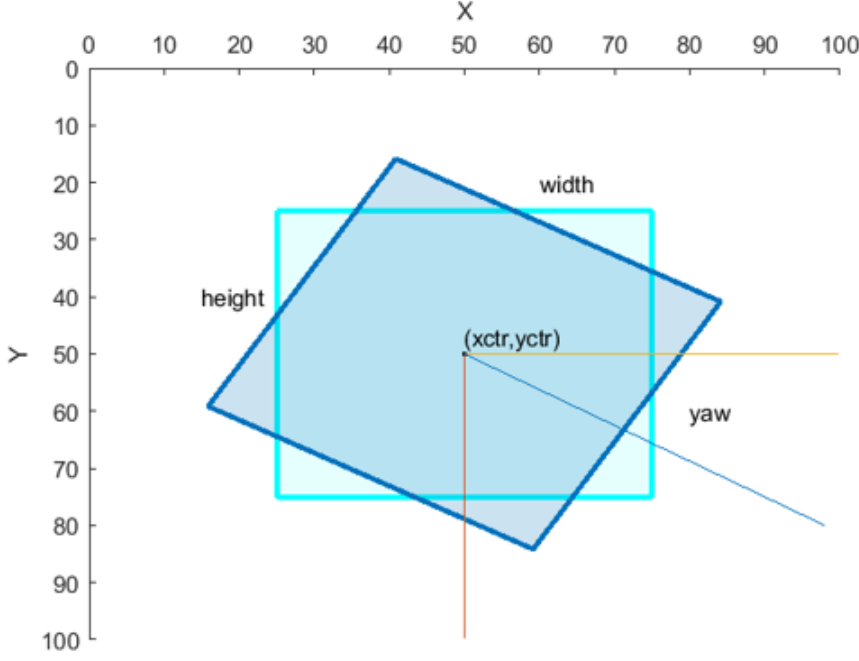
Labeled ground truth images, specified as a datastore. Data must be set up so that calling the datastore with the read and readall functions returns a cell array or table with two or three columns.

When the output contains two columns, the first column must contain bounding boxes, and the second column must contain labels, {boxes,labels}.

When the output contains three columns, the second column must contain the bounding boxes, and the third column must contain the labels, {data,boxes,labels}. In this case, the first column can contain any type of data. For example, the first column can contain images or point cloud data. The last column, labels, must be a cell array that contains M-by-1 categorical vectors containing object class names. All categorical data returned by the datastore must contain the same categories.

The table describes the format of bounding boxes.

Bounding Box	Description
Axis-aligned rectangle	Defined in spatial coordinates as an M-by-4 numeric matrix with rows of the form [x y w h], where: - M is the number of axis-aligned rectangles. - x and y specify the upper-left corner of the rectangle. - w specifies the width of the rectangle, which is its length along the x-axis. - h specifies the height of the rectangle, which is its length along the y-axis.

Bounding Box	Description
Rotated rectangle	Defined in spatial coordinates as an M-by-5 numeric matrix with rows of the form $[xctr\ yctr\ w\ h\ yaw]$, where: M is the number of rotated rectangles. $xctr$ and $yctr$ specify the center of the rectangle. w specifies the width of the rectangle, which is its length along the x-axis before rotation. h specifies the height of the rectangle, which is its length along the y-axis before rotation. yaw specifies the rotation angle in degrees. The rotation is clockwise-positive around the center of the bounding box. 

For more information, see Datastores for Deep Learning (Deep Learning Toolbox).



Note

A pretrained axis-aligned network can be converted to a rotated rectangle network by providing rotated rectangle training data. When you provide the rotated rectangle training data, the `trainYOLOv4objectDetector` function fine tunes the network heads allowing the rotated rectangle detections to occur.



detector — Pretrained or untrained YOLO v4 object detector

yo1ov4objectDetector object

Pretrained or untrained YOLO v4 object detector, specified as a yo1ov4objectDetector object.



options — Training options

TrainingOptionsSGDM object | TrainingOptionsRMSProp object | TrainingOptionsADAM object

Training options, specified as a TrainingOptionsSGDM, TrainingOptionsRMSProp, or TrainingOptionsADAM object returned by the `trainingOptions` (Deep Learning Toolbox) function. To specify the solver name and other options for network training, use the `trainingOptions` function. You must set the `BatchNormalizationStatistics` property of the object as "moving".



checkpoint — Saved detector checkpoint

yo1ov4objectDetector object

Saved detector checkpoint, specified as a yo1ov4objectDetector object. To periodically save a detector checkpoint during training, specify `CheckpointPath`. To control how frequently check points are saved see the `CheckPointFrequency` and `CheckPointFrequencyUnit` training options.

To load a checkpoint for a previously trained detector, load the MAT-file from the checkpoint path. For example, if the `CheckpointPath` property of the object specified by options is 'checkpath', you can load a checkpoint MAT-file by using this code. 'checkpath' is the name of a folder in the current working directory to which the detector checkpoint has to be saved during training.

```
data = load('checkpath/net_checkpoint__19__2021_12_29__01_04_15.mat');
checkpoint = data.net;
```

The name of the MAT-file includes the iteration number and timestamp of when the detector checkpoint was saved. The detector is saved in the `net` variable of the file. Pass this file back into the `trainYOLOv4objectDetector` function:

```
yo1oDetector = trainYOLOv4objectDetector(trainingData,checkpoint,options);
```

Name-Value Arguments

Specify optional pairs of arguments as `Name1=Value1, ..., NameN=ValueN`, where `Name` is the argument name and `Value` is the corresponding value. Name-value arguments must appear after other arguments, but the order of the pairs does not matter.

Before R2021a, use commas to separate each name and value, and enclose `Name` in quotes.

Example: `ExperimentManager="none"` sets the metrics to track with Experiment Manger to "none".



ExperimentManager — Detector training experiment monitoring

"none" (default) | experiments.Monitor object

Detector training experiment monitoring, specified as an `experiments.Monitor` (Deep Learning Toolbox) object for use with the Experiment Manager (Deep Learning Toolbox) app. You can use this object to track the progress of training, update information fields in the training results table, record values of the metrics used by the training, and to produce training plots. For an example using this app, see [Train Object Detectors in Experiment Manager](#).

Information monitored during training:

- Training loss at each iteration.
- Learning rate at each iteration.

Validation information when the training options input contains validation data:

- Validation loss at each iteration.



detector — Trained YOLO v4 object detector

yo1ov4objectDetector object

Trained YOLO v4 object detector, returned as yo1ov4objectDetector object. You can train a YOLO v4 object detector to detect multiple object classes.



info — Training progress information

structure array

Training progress information, returned as a structure array with seven fields. Each field corresponds to a stage of training.

- `TrainingLoss` — Training loss at each iteration. The `trainYOLOv4objectDetector` function uses mean square error for computing bounding box regression loss and cross-entropy for computing classification loss.
- `BaseLearnRate` — Learning rate at each iteration.
- `OutputNetworkIteration` — Iteration number of returned network.
- `ValidationLoss` — Validation loss at each iteration.
- `FinalValidationLoss` — Final validation loss at end of the training.

Each field is a numeric vector with one element per training iteration. Values that have not been calculated at a specific iteration are assigned as NaN. The struct contains `ValidationLoss` and `FinalValidationLoss` fields only when options specifies validation data.

Tips

- To generate the ground truth, use the Image Labeler or Video Labeler app. To create a table of training data from the generated ground truth, use the `objectDetectorTrainingData` function.
- To improve prediction accuracy,
 - Increase the number of images you can use to train the network. You can expand the training dataset through data augmentation. For information on how to apply data augmentation for preprocessing, see [Preprocess Images for Deep Learning](#) (Deep Learning Toolbox).
 - Choose anchor boxes appropriate to the dataset for training the network. You can use the `estimateAnchorBoxes` function to compute anchor boxes directly from the training data.

- A pretrained axis-aligned network can be converted to a rotated rectangle network by providing rotated rectangle training data. When you provide the rotated rectangle training data, the `trainYOLOv4objectDetector` function fine tunes the network heads allowing the rotated rectangle detections to occur.
- When you train a rotated rectangle bounding box detector, use a learning rate approximately one order of magnitude below that of its axis-aligned counterpart training rate.

Version History

Introduced in R2022a

See Also

Apps

[Image Labeler](#) | [Video Labeler](#)

Functions

[trainingOptions](#) (Deep Learning Toolbox) | [yolov4objectDetector](#) | [objectDetectorTrainingData](#) | [trainYOLOv2objectDetector](#)

Objects

Topics

- [Object Detection Using YOLO v4 Deep Learning](#)
- [Estimate Anchor Boxes From Training Data](#)
- [Train Object Detectors in Experiment Manager](#)
- [Anchor Boxes for Object Detection](#)
- [Datastores for Deep Learning \(Deep Learning Toolbox\)](#)